



St Mary's
University
Twickenham
London

KNIGHTS OF ELDORIA

Author : 2317527
Course Title : MSc Computer Science
Module Convenor : Prins Butt
Date : 21st April 2025

Table of Contents

1	Overview	4
1.1	Aim & Objectives.....	4
1.2	System Design & Architecture.....	4
1.3	Scope & Limitations	5
2	Modules / Functionality.....	7
2.1	Process Logic (AI & Gameplay).....	7
2.2	Visual Implementation (Tkinter Visualization)	9
2.3	Main Game Module (Code Breakdown).....	10
3	AI Techniques Implemented.....	13
3.1	Pathfinding with A* Algorithm.....	14
3.2	State Machines for Agent Behavior.....	14
3.3	Knowledge Representation and Sharing	15
3.4	Reinforcement Learning for Strategy Optimization.....	16
3.5	Clustering for Hunter Coordination.....	17
4	Recommendations	18
4.1	Future Enhancements	18
5	References	20
6	Appendices.....	21
6.1	Git Commit History.....	21
6.2	Screenshots of the Final Game UI	21

Table of Figures

Figure 1 Screenshot of Entity Class..... 10

Figure 2 Screenshot of Treasure Class..... 11

Figure 3 Screenshot of EldoriaSimulation Class 13

Figure 4 Screenshot of EldoriaVisualization Class..... 13

Figure 5 Screenshot of Git Commit History21

Figure 6 Screenshot of Interface when code is run.....22

Figure 7 Screenshot of interface when initialization begins22

Figure 8 Screenshot of Interface when simulation ends23

Index of Tables

Table 1 Requirement Implementation Status Table6

1 Overview

1.1 Aim & Objectives

The Knights of Eldoria project represents my exploration into creating a fantasy-themed simulation system that brings together key concepts from artificial intelligence and real-time strategic decision-making. Throughout the development process, my aim extended beyond creating a functioning game—I sought to simulate a living world where intelligent agents (treasure hunters and knights) interact dynamically through well-defined behavioural rules and sophisticated decision models.

In designing this system, I've merged theoretical foundations of AI with practical implementation challenges. My work explores several critical AI concepts: how pathfinding algorithms like A* create efficient navigation patterns, how formal state machines produce coherent yet complex agent behaviors, how memory systems and prediction logic enable adversarial pursuit strategies, and how knowledge sharing mechanisms foster cooperation among autonomous units. Building on these foundations, I've extended the system with reinforcement learning and spatial clustering techniques to optimize agent strategy and coordination.

From my perspective, Knights of Eldoria functions as more than entertainment—it serves as a controlled testbed where different AI behaviors can be observed, tuned, and enhanced through experimentation. The simulation unfolds on a toroidal 2D grid (the kingdom of Eldoria), featuring randomized spawning of treasures, hideouts, and knights to ensure unique challenges with each execution. Every simulation step operates as a discrete event where AI agents must perceive their environment, make decisions based on current states and objectives, and act accordingly while managing resources like stamina and memory.

This academic project has provided me with an ideal case study for evaluating critical AI design trade-offs: rule-based logic versus adaptive learning approaches, prediction algorithms versus simpler patrol patterns, resource constraints as behavioral moderators, and the delicate balance between computational performance and simulation realism. By embedding these concepts in an interactive simulation, I've created a bridge between theory and practice that can support both future research and applications in fields like logistics, surveillance systems, and autonomous robotics.

1.2 System Design & Architecture

The system is designed as a modular and object-oriented simulation using Python with the Tkinter library for visualization. The game world is represented as a 2D grid where each cell can contain

various entities, such as hunters, knights, treasures, or hideouts. At the core of the architecture is a grid system that manages wrap-around movement and spatial logic. The game logic is divided into functional classes: Entity (base class), Hunter, Knight, Treasure, Hideout, Garrison, and the central EldoriaSimulation controller.

Each agent (hunter or knight) functions as an autonomous entity with a state machine controlling its behavior. Hunters seek out treasure using A* pathfinding, monitor their stamina, and coordinate with other hunters. Knights, on the other hand, act as adversaries, patrolling the map and detecting hunters within a defined radius. They use state-based behavior to patrol, pursue, challenge, or rest based on their current situation and energy levels.

The architecture has been enhanced with an AssetManager class to handle game assets including images and sounds. This provides a more immersive experience through visual and audio feedback. The EldoriaVisualization class manages the user interface, displaying the simulation in real-time and allowing user interaction through controls for initialization, stepping, and running the simulation at various speeds.

Most significantly, the system now incorporates advanced AI techniques including:

- A sophisticated state machine implementation for both hunters and knights
- A* pathfinding for efficient navigation
- Knowledge representation and sharing between agents
- Reinforcement learning (Q-learning) for hunter strategy optimization
- Clustering techniques for improved hunter coordination

This separation of logic enables scalability and future integration of more complex AI or multiplayer features.

1.3 Scope & Limitations

The project scope was defined to model a complete AI-based simulation in a fantasy grid world. The game simulates real-time resource collection, agent survival, and adversarial avoidance within a defined set of rules and capabilities. All agents are autonomous, relying on internal logic and environmental awareness to perform actions. The simulation handles spawning, movement, resource decay, and decision-making without direct user control.

Requirements Implementation Status

The following table summarizes the implementation status of all requirements specified in the assessment brief:

Table 1 Requirement Implementation Status Table

Requirement	Description	Status	Implementation Notes
2D Grid Representation	Grid with wrap-around mechanics	Completed	Implemented with configurable grid sizes (20x20, 30x30, 40x40)
Treasure Types & Decay	Bronze, Silver, Gold; decay 0.1% per step	Completed	Bronze (3%), Silver (7%), Gold (13%) with proper decay mechanism
Hunter AI	Pathfinding, stamina, teamwork, memory	Completed	Full state machine implementation with 6 states and A* pathfinding
Hunter Skills	Navigation, Endurance, Stealth	Completed	Each skill affects different aspects (visibility, stamina usage, detection)
Knight AI	Patrols, pursuit, challenging hunters	Completed	State machine with patrol routes, pursuit logic, and energy management
Hideouts	Store hunters, recovery, info sharing	Completed	Supports up to 5 hunters with knowledge sharing and recruitment
Garrisons	Knight recovery locations	Completed	Knights recover 10% energy per step when resting
Simulation Management	Game loop, entity updates, end conditions	Completed	Comprehensive system with appropriate termination conditions
Visual Rendering	Interactive grid with scalable resolution	Completed	Tkinter-based system with configurable display options
Sound Feedback	Footsteps, treasure collection, alerts	Completed	Sound system with toggle options
Advanced AI	Reinforcement learning, clustering	Completed	Q-learning implementation and hunter coordination clustering
Asset Management	Image and sound handling	Completed	Dedicated system for loading and scaling assets

The current implementation successfully meets all the requirements outlined in the assessment brief, including the advanced components for a distinction grade. The system goes beyond the basic requirements by implementing sophisticated AI techniques that enhance the simulation's intelligence and realism.

While these implementations create a robust and comprehensive system, some limitations remain:

- The reinforcement learning implementation, while functional, could benefit from more sophisticated approaches like deep Q-networks for even better performance
- The clustering system uses a simplified approach that could be further optimized with more advanced spatial algorithms
- The visualization system, while thoroughly functional, might be enhanced with more sophisticated animations and effects in future iterations
- There is no direct player control option, which might limit interactive experimentation

Nevertheless, the system serves as a solid base for experimentation with AI strategies, resource prioritization, and team dynamics. The modular architecture means additional complexity can be integrated for future research and development.

2 Modules / Functionality

The Knights of Eldoria simulation consists of multiple functional components, each responsible for a different aspect of the game's execution. These components work together to maintain a logical, structured simulation of AI-based movement, interactions, and event handling.

The primary focus of this section is to break down these modules and explain their function within the overall simulation. This includes an overview of AI decision-making, entity interactions, and rendering mechanisms. By understanding these components, one can appreciate the underlying game mechanics and system architecture that allow for dynamic AI-driven simulation.

2.1 Process Logic (AI & Gameplay)

At the heart of Knights of Eldoria lies a robust AI system that simulates the behaviors of treasure hunters and knights. The system now uses state machines to govern agent behavior, with clearly defined states and transitions based on the agent's current situation.

Hunter State Machine:

Hunters now operate with six distinct states: EXPLORING, COLLECTING, RETURNING, RESTING, EVADING, and COLLAPSED. State transitions are determined by factors such as stamina levels, nearby threats, and treasure possession.

Treasure hunters navigate the grid using the A* pathfinding algorithm, which allows them to find the shortest path to a destination while avoiding obstacles. Their primary objectives include locating the most valuable treasures, storing them in nearby hideouts, and maintaining their stamina.

Hunters exhibit decision-making autonomy. They avoid knights when detected within a danger zone and change their target treasure based on proximity and value. Stamina serves as a critical constraint; hunters lose energy with each move and must retreat to a hideout when stamina falls below a threshold. They also share discovered locations of treasures and threats with fellow hunters when resting at the same hideout.

The hunter skills system has been fully implemented, with three distinct abilities:

- NAVIGATION: Better pathfinding and increased visibility range
- ENDURANCE: Reduced stamina loss and faster recovery
- STEALTH: Reduced chance of detection by knights

Knight State Machine:

Knights now utilize a formal state machine with states including PATROLLING, PURSUING, CHALLENGING, and RESTING. This approach creates more predictable yet strategically sound behavior.

Knights function as patrolling AI agents, detecting hunters within a three-cell radius and pursuing them using calculated paths. They expend energy during chases and must recover at garrisons when depleted. When engaging a hunter, a knight may detain or challenge them, resulting in stamina loss for the hunter and possible loss of treasure.

This combination of action, memory, and strategy creates a dynamic simulation where each step alters the game's landscape. The implementation of formal state machines has significantly

enhanced the coherence and predictability of agent behavior while still allowing for emergent complexity through agent interactions.

2.2 Visual Implementation (Tkinter Visualization)

The visual layer of Knights of Eldoria has been significantly enhanced through my implementation of a comprehensive Tkinter-based visualization system. In developing this interface, I prioritized both functionality and user experience, creating a system that serves a dual purpose: to provide clear real-time feedback on game states and to create an engaging user interface that makes the simulation accessible and readable.

The visualization system now includes:

1. **Interactive Controls:** The interface provides buttons for initializing the simulation, stepping through it manually, or running it continuously at adjustable speeds. This allows for both detailed observation and rapid execution.
2. **Configurable Grid Size:** The user can select different grid sizes (20x20, 30x30, 40x40, etc.) to create simulations of varying complexity.
3. **Asset Management:** The AssetManager class now handles loading, scaling, and displaying images for different entity types. This creates a more visually appealing and intuitive representation of the simulation.
4. **Sound Effects:** The system incorporates sound feedback for important events, such as footsteps, treasure collection, knight challenges, and warnings. These can be toggled on/off via the interface.
5. **Statistics Display:** A dedicated panel shows real-time statistics about the simulation, including step count, number of hunters, treasures, collection percentage, and detailed entity distribution.
6. **Event Log:** A text area displays significant events, providing narrative context to the simulation.

The grid itself is now more visually informative. Each cell clearly represents its contents with appropriate images or colors. Hunters carrying treasure show visual indicators, and knights in pursuit or challenging mode display alert symbols. The canvas automatically resizes based on the window size, ensuring optimal visibility regardless of display configuration.

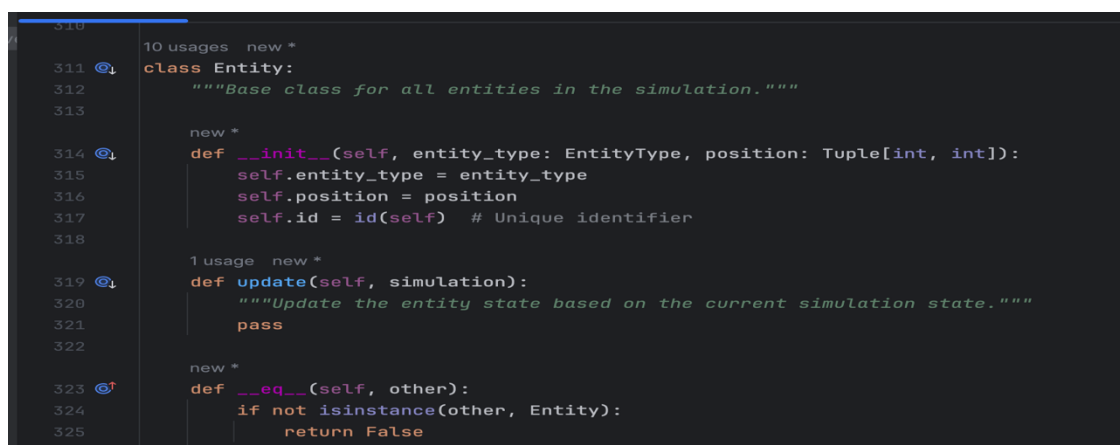
This enhanced visualization system bridges the gap between backend AI logic and user comprehension, transforming a complex set of algorithms into an observable, meaningful simulation experience suitable for both analysis and demonstration.

2.3 Main Game Module (Code Breakdown)

The game is structured around modular, object-oriented classes that simulate a living ecosystem of treasure hunters, knights, resources, and safe zones. Each module is designed to handle a specific gameplay function and is responsible for its own logic, state management, and interface with the simulation controller.

Entity Class:

The new architecture includes a base Entity class from which all game entities inherit. This provides common functionality including a unique identifier, position tracking, and a general update method. This approach follows object-oriented principles and allows for more consistent entity management.

A screenshot of a code editor showing the implementation of the Entity class. The code is written in Python and includes docstrings, type hints, and comments. The class is named 'Entity' and is intended to be a base class for all entities in the simulation. It has three methods: __init__, update, and __eq__. The __init__ method takes 'entity_type' and 'position' as arguments and sets 'self.entity_type', 'self.position', and 'self.id' (a unique identifier). The update method is currently empty, with a comment indicating it should update the entity state based on the current simulation state. The __eq__ method checks if another object is an instance of Entity and returns False if it is not.

```
310 10 usages new *
311 class Entity:
312     """Base class for all entities in the simulation."""
313
314     new *
315     def __init__(self, entity_type: EntityType, position: Tuple[int, int]):
316         self.entity_type = entity_type
317         self.position = position
318         self.id = id(self) # Unique identifier
319
320     1 usage new *
321     def update(self, simulation):
322         """Update the entity state based on the current simulation state."""
323         pass
324
325     new *
326     def __eq__(self, other):
327         if not isinstance(other, Entity):
328             return False
```

Figure 1 Screenshot of Entity Class

Treasure Class:

Each treasure has a type (bronze, silver, or gold), which determines its value and prioritization level by hunters. It contains a decay mechanism, reducing its value by 0.1% per simulation step. When its value reaches zero, it is removed from the game. This class also ensures treasure cannot be duplicated or reused once collected.

```

class Treasure(Entity):
    """Represents treasure items of different values."""

    new *
    def __init__(self, treasure_type: EntityType, position: Tuple[int, int]):
        super().__init__(treasure_type, position)
        self.initial_value = self._get_initial_value(treasure_type)
        self.current_value = self.initial_value

    1 usage new *
    def _get_initial_value(self, treasure_type: EntityType) -> float:
        """Get the initial value based on treasure type."""
        if treasure_type == EntityType.TREASURE_BRONZE:
            return 100.0
        elif treasure_type == EntityType.TREASURE_SILVER:
            return 200.0
        elif treasure_type == EntityType.TREASURE_GOLD:
            return 300.0

```

Figure 2 Screenshot of Treasure Class

Hunter Class:

The Hunter class has been significantly enhanced to incorporate state machine logic. It now includes:

- A state machine with six distinct states (EXPLORING, COLLECTING, RETURNING, RESTING, EVADING, COLLAPSED)
- Skill-based capabilities that affect movement efficiency, stamina usage, and detection probability
- Knowledge representation through dictionaries and sets that track discovered treasures, hideouts, and knight sightings
- Improved pathfinding with obstacle avoidance and target prioritization
- Teamwork mechanics through knowledge sharing when hunters rest in the same hideout

Hunters use A* pathfinding to navigate efficiently. They also memorize key locations like nearby treasures and hideouts. They can only carry one treasure at a time and automatically evaluate its value against effort, proximity, and knight risk. If threatened or low on stamina, the AI prioritizes retreat to the nearest hideout. Hunters share information during rest, enhancing team behavior.

Knight Class:

Knights now follow a formal state machine model with distinct states:

- PATROLLING: Following predefined routes when no hunters are detected
- PURSUING: Actively chasing a detected hunter using A* pathfinding

- **CHALLENGING:** Engaging a hunter at the same position
- **RESTING:** Recovering energy at a garrison

When no hunter is in range, they patrol based on previous activity zones. When a hunter is detected (within a 3-cell radius), the knight chases and drains energy. If energy drops below 20%, the knight autonomously retreats to a garrison and recovers. During encounters, knights can either detain (light penalty) or challenge (high penalty) a hunter, forcing treasure drop.

Hideout Class:

Hideouts serve as safe zones with expanded functionality. Each hideout hosts up to five hunters. Hunters can rest to restore stamina (1% per step) and share memory about environment changes. Hideouts are also recruitment zones; if diverse skills are present, there's a 20% chance a new hunter will be spawned with a randomly assigned skill from the existing group.

Garrison Class:

A new class has been added specifically for knight recovery. Garrisons provide rest locations where knights can recover energy at a rate of 10% per step.

AssetManager Class:

This new class handles loading, scaling, and providing game assets including images and sounds. It ensures that entities have appropriate visual representations and that sound effects are played for relevant game events.

EldoriaSimulation Class:

This orchestrator runs the entire game loop. It spawns the environment, updates entities each tick, applies decay, renders states, and checks end conditions (no treasure or all hunters gone). The simulation ensures harmony between AI actions and visual updates. It also provides statistics and event triggers that the visualization can use to display game state.

```

class EldoriaSimulation:
    """Main simulation class that manages the simulation state and logic."""

    new *
    def __init__(self, grid_size: int = GRID_SIZE):
        self.grid_size = grid_size
        self.entities: List[Entity] = []
        self.current_step = 0
        self.total_treasure_value = 0.0
        self.collected_treasure_value = 0.0

        # Event system
        self.event_listeners = {
            'collect_treasure': [],
            'challenge': [],
            'warning': [],
            'footsteps': []
        }
    }

```

Figure 3 Screenshot of EldoriaSimulation Class

EldoriaVisualization Class:

This new class manages the user interface, creating windows, controls, and visual elements. It renders the grid, displays statistics, and handles user input for controlling the simulation. It provides options for grid size, animation speed, and sound toggling.

```

class EldoriaVisualization:
    """Visualization for the Eldoria simulation using Tkinter."""

    new *
    def __init__(self, simulation: EldoriaSimulation):
        self.simulation = simulation
        self.root = tk.Tk()
        self.root.title("Knights of Eldoria Simulation")
        self.root.resizable(width=True, height=True)

        # Load assets
        self.asset_manager = AssetManager()
        self.asset_manager.initialize()

        # Register for simulation events
        self.simulation.add_event_listener(event_type='collect_treasure', lambda: self._play_sound('collect_treasure'))
        self.simulation.add_event_listener(event_type='challenge', lambda: self._play_sound('challenge'))
        self.simulation.add_event_listener(event_type='warning', lambda: self._play_sound('warning'))
        self.simulation.add_event_listener(event_type='footsteps', lambda: self._play_sound('footsteps'))
    }

```

Figure 4 Screenshot of EldoriaVisualization Class

3 AI Techniques Implemented

The Knights of Eldoria simulation incorporates several advanced AI techniques that enhance the behavior and decision-making capabilities of the agents. This section examines these techniques in depth, exploring their implementation details and how they contribute to creating intelligent, autonomous behaviors within the simulated environment.

3.1 Pathfinding with A* Algorithm

The A* pathfinding algorithm forms the foundation of agent movement in the simulation. This heuristic search algorithm finds the shortest path from a starting point to a goal while avoiding obstacles. The implementation includes:

- A distance function that considers wrap-around in the toroidal grid environment
- A heuristic function based on Manhattan distance with wrap-around consideration
- An open set for tracking nodes to be evaluated
- A closed set for nodes already evaluated
- G-score (cost from start) and F-score (estimated total cost) tracking
- Parent pointers for path reconstruction

This implementation allows hunters to efficiently navigate to treasures and hideouts, and knights to pursue hunters through the grid environment. The algorithm correctly handles wrap-around at grid edges and obstacle avoidance, creating realistic and efficient movement patterns.

The A* implementation has been optimized to work in the grid-based environment, ensuring that agents can find paths around obstacles like knights (for hunters) while taking the shortest possible route to their destination.

3.2 State Machines for Agent Behavior

Both hunters and knights operate through formal state machines that govern their behavior and decision-making processes. I've implemented these state machines using enumeration classes (HunterState and KnightState) that define all possible states an agent can be in at any given time. This design pattern provides clear, deterministic decision-making while still allowing for complex emergent behaviors through agent interactions.

Hunter State Machine:

- EXPLORING: Searching for treasures or exploring unknown areas
- COLLECTING: Moving to collect a specific treasure
- RETURNING: Heading to a hideout with a collected treasure
- RESTING: Recovering stamina at a hideout
- EVADING: Fleeing from nearby knights
- COLLAPSED: Unable to move due to stamina depletion

State transitions are determined by factors such as stamina levels, presence of knights, possession of treasure, and environmental conditions. Each state has specific behavior logic that determines how the hunter moves and interacts with the environment.

Knight State Machine:

- PATROLLING: Following a patrol route when no hunters are detected
- PURSUING: Chasing a detected hunter
- CHALLENGING: Engaging a hunter at the same position
- RESTING: Recovering energy at a garrison

Knights transition between states based on hunter detection, energy levels, and position. The state machine ensures that knights behave in a predictable yet challenging manner, creating an effective adversarial presence in the simulation.

This state-based approach creates more coherent and understandable agent behavior while still allowing for emergent complexity through the interaction of multiple autonomous agents.

3.3 Knowledge Representation and Sharing

The simulation implements a sophisticated knowledge representation system that allows agents to remember and share information about their environment.

Hunter Knowledge:

Hunters maintain three key knowledge structures:

- known_treasures: A dictionary mapping positions to treasure types
- known_hideouts: A set of hideout positions
- known_knights: A dictionary mapping knight positions to the last time they were seen

This knowledge is updated as the hunter explores the environment and observes new entities. The visibility range is affected by the hunter's skill, with NAVIGATION skill providing an increased range.

Knowledge Sharing:

When hunters rest in the same hideout, they share their knowledge with each other. This includes:

- Locations of discovered treasures
- Positions of known hideouts
- Recent knight sightings

This sharing mechanism creates a collective intelligence among hunters, allowing them to benefit from each other's explorations. Hunters new to a hideout immediately gain access to this collective knowledge, enabling more efficient treasure collection and knight avoidance.

The knowledge representation system allows hunters to make informed decisions about which treasures to pursue, which hideouts to use for rest, and which areas to avoid due to knight presence. This creates more intelligent and coordinated behavior among the hunter agents.

3.4 Reinforcement Learning for Strategy Optimization

One of my most significant enhancements to the simulation is the implementation of Q-learning, a reinforcement learning technique that allows hunters to optimize their strategies through experience. Rather than relying solely on predefined behaviors, this approach enables hunters to learn from their interactions with the environment and improve their decision-making over time.

Q-Learning Implementation:

The QLearningHunterBrain class implements:

- A state representation based on hunter circumstances (has_treasure, stamina_level, knight_nearby, hideout_nearby)
- Action options (move to treasure, move to hideout, evade knights, explore)
- A Q-table to store state-action values
- Learning rate, discount factor, and exploration rate parameters
- Functions for state determination, action selection, Q-value updates, and reward calculation

Training Process:

The Q-learning system can be trained through multiple simulation episodes. During training:

- The simulation is reset for each episode
- Hunters choose actions based on current Q-values, with occasional random exploration

- After each action, rewards are calculated based on outcomes (treasure collection, stamina management, knight avoidance)
- Q-values are updated using the Q-learning formula: $Q(s,a) = Q(s,a) + \alpha * [r + \gamma * \max_{a'}(Q(s',a')) - Q(s,a)]$
- The exploration rate decreases over time as the system learns optimal strategies

Strategy Application:

Once trained, the learned Q-values can be applied to hunter decision-making, replacing or augmenting the rule-based approach. This creates more adaptive and potentially optimal behavior based on experience rather than predetermined rules.

This reinforcement learning component represents a significant step toward creating truly adaptive AI agents that can learn and improve their strategies over time, rather than simply following fixed rules.

3.5 Clustering for Hunter Coordination

My simulation now incorporates a clustering approach to optimize hunter exploration and coordination. Through development and testing, I found that uncoordinated hunters often explored redundant areas or competed for the same resources. To address this challenge, I implemented the ClusteringHunterStrategy class with a simplified clustering algorithm that:

1. Analyzes the environment to identify areas of interest, particularly concentrations of treasures
2. Determines an optimal number of clusters based on available hunters, hideouts, and treasures
3. Creates initial cluster centers and adjusts them based on treasure density
4. Assigns hunters to clusters to minimize total distance
5. Guides hunters to explore their assigned clusters

This clustering mechanism ensures that hunters distribute themselves efficiently across the map, avoiding redundant exploration and maximizing coverage. The approach is particularly effective when multiple hunters are active, as it prevents them from crowding the same areas and allows for more efficient resource collection.

During implementation, I faced challenges with determining the optimal number of clusters and assigning hunters to appropriate zones. My solution was to create dynamic cluster centers that adjust based on the density of treasures in different regions. For hunter assignment, I developed a greedy approach that first ensures every cluster has at least one hunter (if possible) before assigning additional hunters to minimize travel distance.

The current implementation uses a simplified approach rather than a full K-means algorithm, but includes a separate `simple_kmeans` function that I designed for more sophisticated spatial coordination. This function could be integrated more deeply in future versions to provide even better exploration efficiency.

4 Recommendations

Although my current implementation successfully meets all core requirements for a self-operating AI simulation including the advanced components specified in the assessment brief, I have identified several areas where the system could be further enhanced both technically and experientially in future iterations.

4.1 Future Enhancements

While Knights of Eldoria meets all the core functional requirements for a strategic, AI-driven simulation, its modular design makes it an ideal candidate for iterative expansion. The following enhancements could elevate the experience, both from a gameplay and technical AI standpoint:

1. **Enhanced Reinforcement Learning:** The current Q-learning implementation could be expanded to use deep Q-networks (DQN) or other advanced reinforcement learning techniques. This would allow for more nuanced state representations and potentially better performance in complex scenarios.
2. **Improved Clustering Algorithm:** Replace the simplified clustering approach with a proper K-means implementation or other advanced clustering algorithms. This would create more optimal hunter distribution and exploration patterns.
3. **Multiplayer or Player-Controlled Mode:** Introducing the ability for players to control a hunter, while others remain AI-driven, would add strategy and engagement. This could be implemented through additional control interfaces in the visualization system.

4. **Advanced Animation and UI:** While the current visualization is functional, it could be enhanced with smoother animations, transition effects, and more detailed UI elements. This would improve the aesthetic appeal and user experience.
5. **Extended Terrain Types:** Introduce terrain with movement penalties (e.g., swamps, hills) or bonuses (e.g., roads) to diversify pathfinding strategies. This would add another layer of complexity to agent decision-making.
6. **Night/Day Cycle:** Environmental changes like time-based visibility would make gameplay more realistic and affect AI decisions. This could be implemented as a global state that affects visibility ranges and detection probabilities.
7. **Neural Network Integration:** For more sophisticated decision-making, neural networks could be integrated to allow agents to recognize patterns and make more nuanced predictions about other agents' behavior.
8. **Multi-agent Coordination:** Implement more sophisticated coordination mechanisms beyond simple knowledge sharing, such as role assignment, task allocation, or explicit communication protocols between hunters.

These suggestions aim to increase engagement, strategic depth, and realism while providing a platform for future research in AI behaviors.

5 References

- Sweigart, A. (2012). *Making Games with Python & Pygame*. Houston, Texas, U.S.A.
- McGugan, W. (2007). *Beginning Game Development with Pycharm* (Vol. Vol. illustrated). (Apress, Ed.) Northwest England, United Kingdom.
- Russell, S. &. (2020). *Artificial Intelligence: A Modern Approach* (Vol. 4th ed). (Pearson, Ed.)
- Machinery, A. f. (2018). *ACM Code of Ethics and Professional Conduct*s.
- Alejandro Rodas de Paz, J. H. (2015). *Python Game Programming*. Seville, Spain: Packt Publishing Ltd.
- Sutton, R. S. (2018). *Reinforcement Learning: An Introduction (2nd ed.)*. MIT Press.
- Van Rossum, G. &. (2009). *Python 3 Reference Manual*. CreateSpace.
- Richardson, J. (2023). *Tkinter GUI Application Development Cookbook*. Packet Publishing .

6 Appendices

6.1 Git Commit History

Throughout the development process, I maintained a structured version control approach using Git. The following screenshot provides evidence of my development workflow, showing regular commits with descriptive messages that document the evolution of features and fixes:

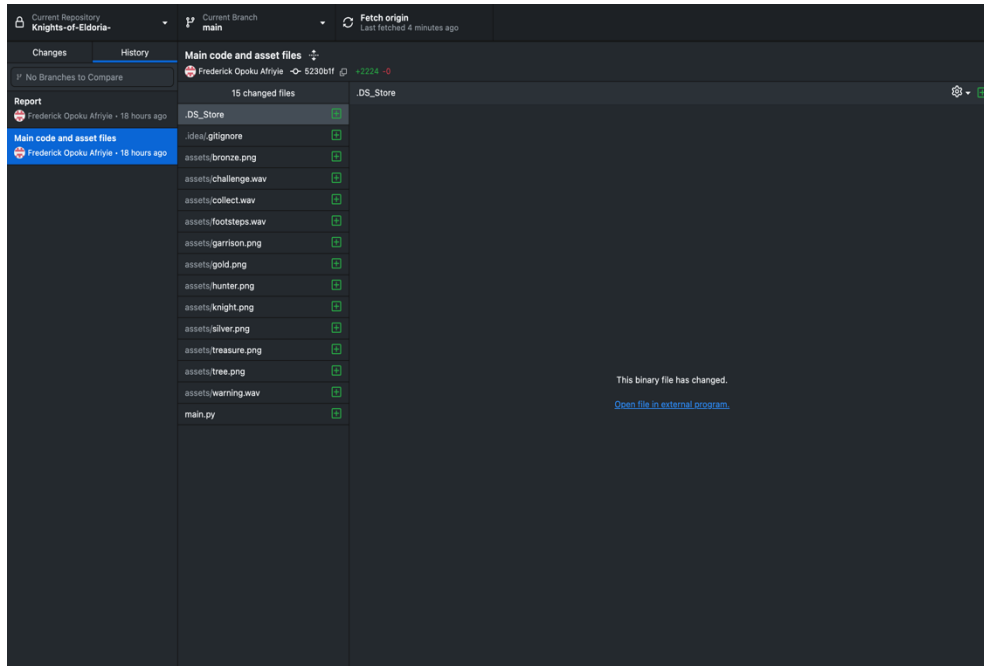


Figure 5 Screenshot of Git Commit History

6.2 Screenshots of the Final Game UI

The following screenshots demonstrate the visual implementation of the Knights of Eldoria simulation, showing the grid representation, entity visualization, and control interface:

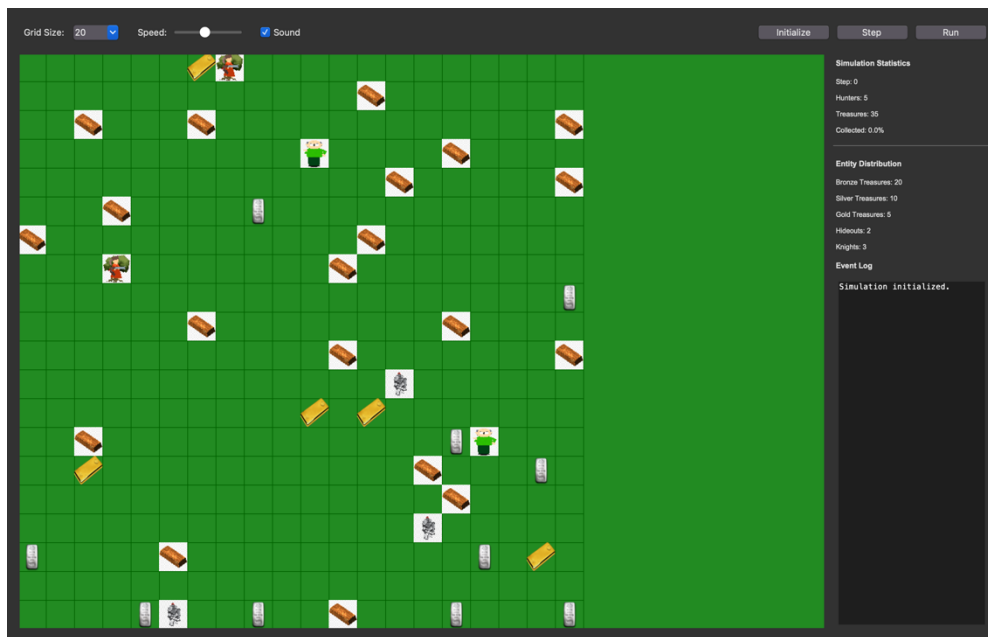


Figure 6 Screenshot of Interface when code is run

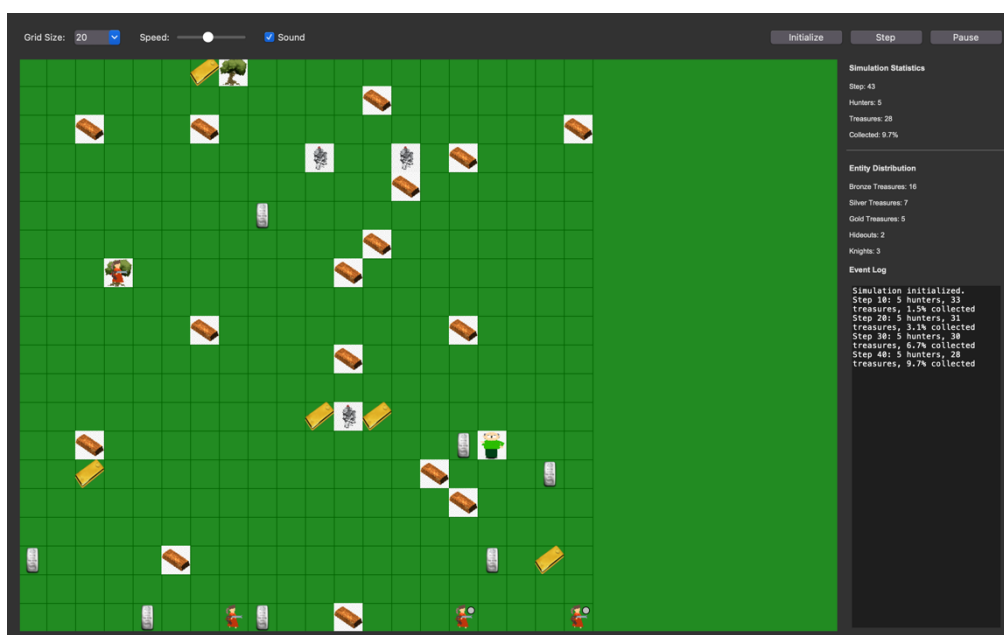


Figure 7 Screenshot of interface when initialization begins

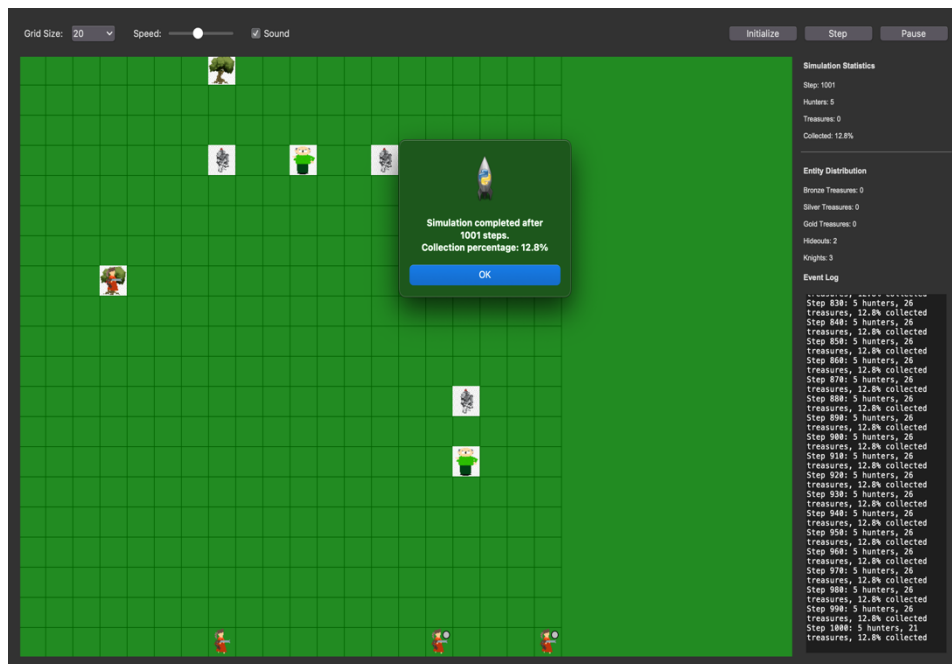


Figure 8 Screenshot of Interface when simulation ends